Ship Real AI

Ship Real AI

The AI Engineer Bootcamp for LLMs, RAG, MCP,
AI Agents and Evals, from First Principles
to Production and Interviews

Sridhar Mukkandi

Draft

Ship Real AI: The AI Engineer Bootcamp for LLMs, RAG, MCP, AI Agents and Evals, from First Principles to Production and Interviews

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles.

Independently published.

About the examples in this book. Tarkika, its customers and its staff are imagined. Every number about Tarkika, and every price marked as an example, is assumed, to explain the ideas clearly; none is a measurement of a real company or product. Facts about real companies, products and incidents are cited.

Products and services change quickly. Model names, versions and prices were checked on the date given in the version register at the back of the book. All trademarks belong to their respective owners.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or financial advice.

Text set in IBM Plex Serif, Work Sans and IBM Plex Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vi

I The job and the foundations vii

1 Python for AI engineers 1

Appendices 8

References 8

About the author 9

Preface

The preface is written last, once the modules are final.

PART I

The job and the foundations

The first modules give you the ground the rest of the book stands on: the Python habits that keep model calls safe and fast, what an AI engineer builds around a model, and the parts of probability, machine learning and language processing you will actually use at work. None of it is research-level. If an idea is here, a later module or a real job needs it.

01

Python for AI engineers

Ravi is a developer at Tarkika. He writes a short Python script that reads customer messages from a file, sends each one to a language model, and asks for one word back: *refund*, *delivery*, *product* or *other*. He tries it on ten messages. All ten come back right.

Then the Diwali sale starts, and he runs it on one day of mail, 4,000 messages.

1. The progress counter says the run will take more than three hours.
2. Two hours in, at message 2,180, the script stops with `429 Too Many Requests`. Nothing was saved.
3. When he reruns a small batch, the replies are not always one word. Sometimes "Refund". Sometimes "refund." with a full stop. Once, "This customer seems to want their money back." His code was waiting for exactly `refund`.
4. A week later, a small change breaks the script, and nobody notices for three days.

The model was not the problem. Ravi wrote the script as if calling a model were like calling any other function: text goes in, text comes out. It looks like that. It does not behave like that. A normal function is fast, always works, and gives the same output for the same input. A model call is slow, sometimes fails, and gives a different output for the same input. Each of Ravi's four problems comes from one of these differences, and this module takes them one by one.

1.1 The same input does not give the same output

Ravi's code is strict. For Python, `"Refund" == "refund"` is false. The model is not strict. It writes language, and language allows the same meaning in many forms. Ask it the same question five times and you can get five different sentences, all meaning the same thing.

So something has to stand between the two, and check each reply before the strict side uses it.

Before reading on: what exactly would you check in each of Ravi's replies?

The category must be one of the four allowed words. If the customer mentioned an order number, it should be there. If Ravi also wants to know whether the message is urgent, that must be a plain true or false. This list is a description of the shape Ravi expects. Write it as code, and every reply can be checked against it automatically. Such a description is called a **data model**. In Python, most AI projects write it with a library called Pydantic:

```
from typing import Literal
from pydantic import BaseModel, ValidationError

class Ticket(BaseModel):
    category: Literal["refund", "delivery", "product", "other"]
    order_id: str | None = None
    urgent: bool

reply = '{"category": "Refund request", "urgent": true}'
try:
    ticket = Ticket.model_validate_json(reply)
except ValidationError as e:
    print(e.errors()[0]["msg"])
```

Each line of `Ticket` is one rule. `Literal[...]` allows only the four words. `str | None = None` means the order number is text, or missing. `bool` means true or false. The reply here is in JSON, a common text format for data, and the model wrote "Refund request" as the category. The code prints:

```
Input should be 'refund', 'delivery', 'product' or 'other'
```

The bad reply is stopped at the door, with the reason, instead of breaking something three functions later.

Notice the notes such as `str | None` and `bool`. They are **type hints**. In ordinary Python, they are only notes for people and code editors, and Python does not enforce them. Pydantic enforces them. In AI work they have one more reader, and it is the model itself.

```
def check_order_status(order_id: str) -> str:
    """Return the delivery status of one Tarkika order.

    order_id: the order number, for example TK-48213.
    """
```

When you give a function like this to a model as a **tool**, a function the model can ask your program to run, the library you use reads the function's name, type hints and docstring. It turns them into a description and sends that description to the model

(OpenAI 2025). The model decides when to call the tool, and with what, from that text alone. A careless docstring is therefore not a style problem. It is a wrong instruction to the model. We come back to this in Module 7.

Checking the reply solves problem 3. It cannot help when there is no reply at all, which is problem 2.

1.2 A model call can fail

A call to a model travels over the internet to someone else's computers, and some calls fail. For each failure, ask one question: if I wait and try again, will it work?

- **429, too many requests.** Ravi sent more calls than his account allows in a short time. Waiting fixes it.
- **500 or 529, the server has a problem or is overloaded.** Waiting usually fixes it.
- **No reply in time.** Trying again usually works.
- **400, bad request.** Something in the call is wrong, such as text that is too long. Waiting never fixes it.
- **401, unauthorised.** The API key is wrong. Waiting never fixes it.

So retry only the first three. Retrying the others sends the same mistake again.

Before reading on: after a 429, how long should the script wait before trying again?

If it tries again at once, the server is still busy, and the call fails again. Now imagine a thousand scripts, each told to wait exactly one second. They all come back at the same moment and flood the server again. Two changes fix this. Wait longer after each failure: up to 1 second, then 2, then 4, then 8. And pick a random wait within that limit, so that different scripts come back at different times. This is called **exponential backoff with jitter**: exponential because the limit doubles each time, jitter for the randomness (Brooker 2015). After a fixed number of tries, stop and record the failure.

```python
import random, time

RETRYABLE = {429, 500, 503, 529}

def call_with_retry(send, max_attempts=5):
    for attempt in range(max_attempts):
        if attempt > 0:
            time.sleep(random.uniform(0, 2 ** (attempt - 1)))   # full jitter
        status, body = send()
        if status == 200:
            return body
        if status not in RETRYABLE:
            raise RuntimeError(f"Not retryable: {status}")
    raise RuntimeError("Gave up after retries")
```

send is any function that makes one call and returns the status code and the reply. Figure 1.1 shows the same logic as a picture.

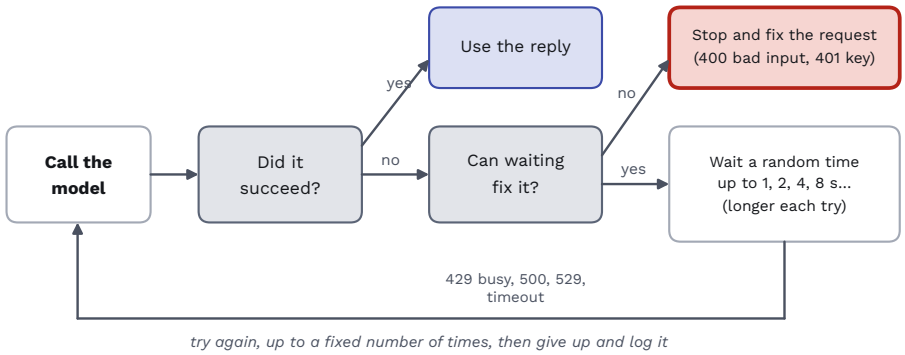


Figure 1.1 Only failures that waiting can fix are tried again, and each wait is longer and partly random.

Five attempts have four waits, of at most 1, 2, 4 and 8 seconds. In the worst case the script waits 15 seconds before giving up on a message. For a batch that runs at night, that is fine. For a customer waiting in a chat window, it is too long, and two attempts are better. The right number depends on who is waiting.

One more thing to check. Most official client libraries for model providers already retry a failed call a few times on their own. If the library tries 3 times and your loop tries 5 times, one message can be sent 15 times.

Retries make crashes rare. They do not make them impossible, and Ravi still lost two hours because his script kept all results in memory until the end. Write each result to

a file the moment it arrives. When the script starts, it reads the file and skips what is already done. Then a crash at message 2,180 costs one message, not two hours.

The script no longer loses work. It is still slow.

1.3 A model call is slow, but your computer is not busy

Each call takes about 3 seconds. One after another, 4,000 calls take 12,000 seconds: 3 hours and 20 minutes.

Before reading on: during those 3 seconds, how much work is Ravi's computer doing?

Almost none. It sends a few kilobytes of text, then waits. The work happens on the provider's machines. So for three hours, Ravi's program mostly waits.

Think of someone in Tarkika's support team who emails a courier about one lost parcel, and then watches the inbox until the courier replies before writing about the next parcel. Nobody works like that. You send twenty emails, and handle the replies as they come. Python can do the same:

```
import asyncio

async def ask_model(message):          # stand-in for a real model call
    await asyncio.sleep(3)             # the time spent waiting for the reply
    return "refund"

async def classify_all(messages, limit=20):
    gate = asyncio.Semaphore(limit)
    async def one(m):
        async with gate:
            return await ask_model(m)
    return await asyncio.gather(*(one(m) for m in messages))

results = asyncio.run(classify_all(["paisa wapas karo"] * 100))
```

An `async` function can pause while it waits, and let other calls run in the meantime. `gather` starts all the calls. The **semaphore** lets only 20 be in progress at once. These 100 messages finish in about 15 seconds instead of 300. Doing many waiting tasks at the same time like this is called **concurrency** (Figure 1.2).

Then why not start all 4,000 at once? Because the provider allows each account only so many calls per minute. Work it out for 20 at a time:

1. Each slot finishes one call every 3 seconds, so 20 calls a minute.
2. Twenty slots finish 20 × 20 = 400 calls a minute.

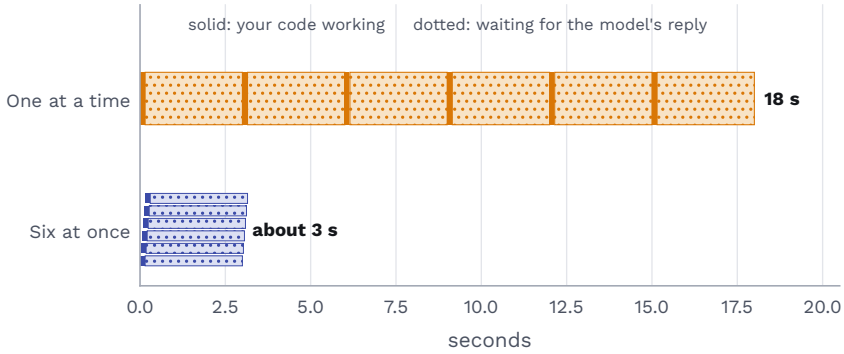


Figure 1.2 Six calls of 3 seconds. One at a time, they take 18 seconds, almost all of it waiting. Started together, the waits overlap and the batch takes about 3 seconds.

3. If Tarkika's account allows 100 calls a minute, 4,000 messages need at least 40 minutes, whatever the code does.

Go above the limit and you get 429 errors, which you retry, which adds even more calls. So the number of calls at once is set by the account's limit, not by how fast you want to finish. Here, 5 at a time gives exactly 100 a minute.

Three hours became forty minutes, and the script survives errors. Problem 4 is left.

1.4 You cannot test a model call like a function

To catch a silent break, you write tests. A normal test gives a function a fixed input and checks for one exact output. Try that here: send "paisa wapas karo" and check that the answer is `refund`. It passes today. Tomorrow the model says "Refund", and the test fails, though nothing in your code changed. A test that fails at random is soon ignored.

Before reading on: which parts of Ravi's script give the same result every time?

Everything Ravi wrote: reading the file, checking the shape, retrying, saving. Only the model's reply changes. So test the two parts separately.

1. **Ravi's own code** is tested with a fake model: a few lines that return replies you choose, including broken ones and errors. Every run gives the same result, needs no network and costs nothing.
2. **The model** is checked on a set of real messages, with checks like "is the category one of the four?" The result is a score, not a pass or fail. These checks are called **evals**, and Module 11 is about them.

```
import pytest
from triage import parse_ticket, BadReply   # parse_ticket checks a reply with Ticket

def test_valid_reply():
    t = parse_ticket('{"category": "delivery", "urgent": false}')
    assert t.category == "delivery"

def test_reply_that_is_not_json():
    with pytest.raises(BadReply):
        parse_ticket("Sure! This looks like a delivery issue.")
```

1.5 What changed

Figure 1.3 shows Ravi's script after the four fixes.

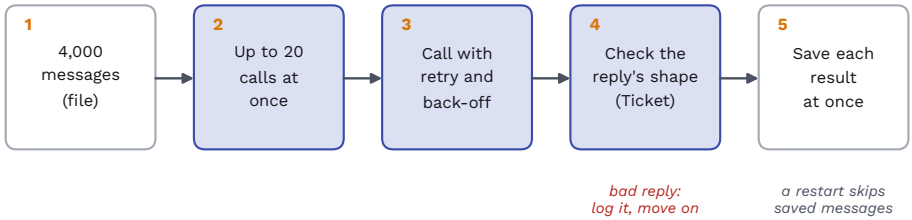


Figure 1.3 Ravi's script, rebuilt: up to 20 calls at once, retries for failures that waiting can fix, every reply checked, and each result saved as soon as it arrives.

Ravi did not need a better model. He needed to stop treating the model like an ordinary function. It is slow, so he runs calls together. It can fail, so he retries carefully and saves as he goes. It answers differently each time, so he checks every reply and tests his own code apart from the model.

One difference was left out: every call also costs money. Module 2 looks at what the model actually is, what an AI engineer builds around it, and what each of those 4,000 calls costs.

References

Brooker, Marc. 2015. *Exponential Backoff and Jitter*. AWS Architecture Blog. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/.

OpenAI. 2025. *OpenAI Agents SDK: Function Tools*. Documentation. https://openai.github.io/openai-agents-python/tools/.

About the author

Sridhar Mukkandi is an applied AI engineer who builds AI agents and the decision systems behind them. He writes for engineers who want AI systems they can trust in production.

sridhar.authorhub@gmail.com